

Documentazione del Progetto di Basi di Dati

Lafragola Daniele - N86005559

27 gennaio 2026

Indice

1	Introduzione	2
1.1	Descrizione del Progetto	2
2	Modello Concettuale	2
2.1	Class Diagram	3
2.2	Ristrutturazione	4
2.2.1	Eliminazione delle ridondanze	4
2.2.2	Eliminazione delle generalizzazioni/specializzazioni	4
2.2.3	Analisi degli attributi multivalore	5
2.2.4	Analisi degli attributi strutturati	5
2.2.5	Analisi delle chiavi primarie	5
2.3	Class Diagram Ristrutturato	6
3	Dizionari	8
3.1	Dizionario delle Classi	8
3.2	Dizionario delle Associazioni	10
3.3	Dizionario dei Vincoli	12
4	Modello Logico	13
4.1	Schema Logico	13
5	Progettazione Fisica	13
5.1	Definizioni delle Tabelle	13
5.1.1	Tabella Utente	14
5.1.2	Tabella Passeggero	14
5.1.3	Tabella Prenotazione (con Trigger)	14
5.1.4	Tabella Volo Nazionale	16
5.1.5	Tabella Volo Internazionale	17

5.1.6	Tabella Gate	17
5.2	Trigger e altre Funzioni	18
5.2.1	Trigger per la gestione delle prenotazioni di un volo cancellato	18

1 Introduzione

Questo elaborato documenta la progettazione e la realizzazione di una base di dati relazionale del DBMS PostgreSQL. Ad opera dello studente Lafragola Daniele, del Corso di Laurea in Informatica presso l'Università degli Studi di Napoli Federico II. Il database ha lo scopo di rappresentare e gestire un sistema informativo aeroportuale.

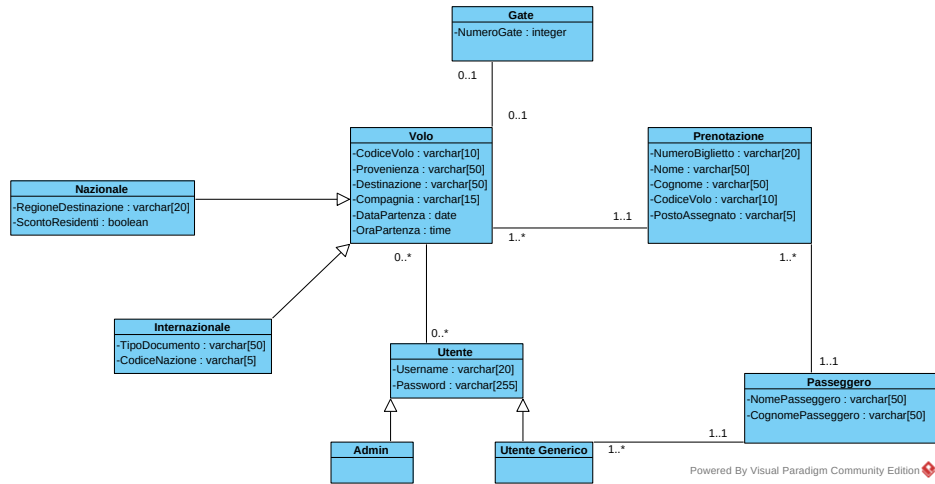
1.1 Descrizione del Progetto

Esso permette l'accesso a diverse tipologie di utenti, come utenti facenti parte dello staff amministrativo, che possono visualizzare e gestire informazioni sui voli, le prenotazioni e i gate, e utenti comuni/passeggeri, che possono visualizzare e prenotare un volo o più voli. Il sistema inoltre, divide i voli tra voli nazionali e internazionali. Il sistema tiene traccia dei passeggeri registrati, delle loro prenotazioni effettuate, dei voli previsitati e dei gate associati alla partenza dei voli.

2 Modello Concettuale

Il modello concettuale è stato realizzato utilizzando il diagramma E-R (Entità-Relazione).

2.1 Class Diagram



2.2 Ristrutturazione

Il precedente Class Diagram andrà ora ristrutturato per permettere la sua implementazione. La ristrutturazione seguirà questi passaggi:

- Eliminazione delle ridondanze
- Eliminazione delle generalizzazioni/specializzazioni
- Analisi degli attributi multivalore
- Analisi degli attributi strutturati
- Analisi delle chiavi primarie

2.2.1 Eliminazione delle ridondanze

Le ridondanze rappresentano degli attributi che possono essere facilmente ricavabili o calcolati da altri presenti. Nello schema, le entità Prenotazione e Passeggero condividono attributi del passeggero, come Nome e Cognome. Nella entità Prenotazione, dunque, queste informazioni sono ridondanti perché già presenti nell'entità Passeggero. La soluzione più corretta, è quella di eliminare gli attributi ridondanti dall'entità Prenotazione.

2.2.2 Eliminazione delle generalizzazioni/specializzazioni

Le generalizzazioni rappresentano delle entità (Figlie) che ereditano attributi e relazioni da un'altra entità (Padre). Nel nostro caso, sono presenti due diverse generalizzazioni/specializzazioni:

- La specializzazione dell'entità Utente, in Admin e Utente Generico.
- La specializzazione dell'entità Volo, in Volo Nazionale e Volo Internazionale.

Per l'eliminazione di queste generalizzazioni, si è deciso di adottare due strategie differenti:

1. Per la specializzazione dell'entità Utente, la scelta migliore è quella dell'accorpamento. In questo modo, si manterranno tutte le informazioni in un'unica tabella, aggiungendo un attributo di tipo booleano per distinguere tra Admin e Utente Generico.
2. Per la specializzazione dell'entità Volo, si è deciso di adottare la strategia del partizionamento. In questo modo, saranno presenti due entità distinte per i voli nazionali e quelli internazionali, riducendo anche il possibile numero di operazioni richieste.

2.2.3 Analisi degli attributi multivalore

Gli attributi multivalore sono quegli attributi che possono assumere più di un valore per una singola istanza dell'entità. Nello schema, non sono presenti attributi multivalore, per tanto non è necessario effettuare alcuna operazione.

2.2.4 Analisi degli attributi strutturati

Gli attributi strutturati sono quegli attributi che posseggono più componenti. Anche in questo caso, non sono presenti attributi strutturati nello schema.

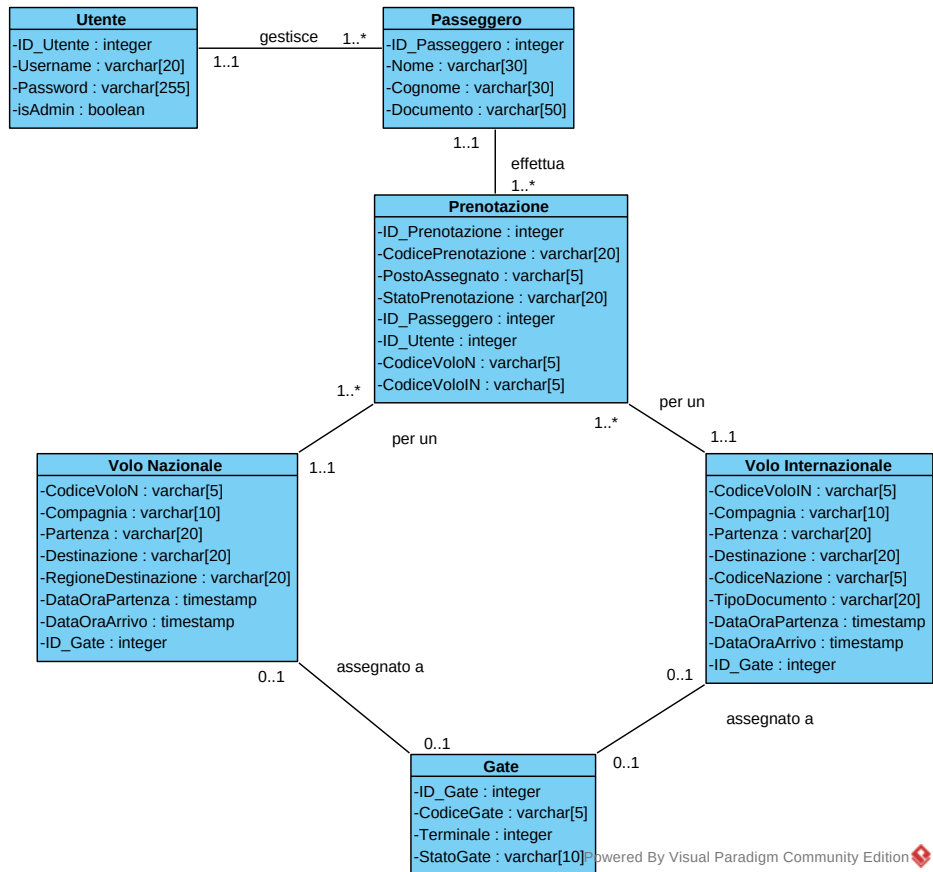
2.2.5 Analisi delle chiavi primarie

Nello schema, sono presenti entità ai quali è consigliabile aggiungere una chiave primaria "surrogata", ovvero un attributo composto da un intero, che identifica quella specifica istanza dell'entità. Queste entità sono:

- Utente → Aggiunta dell'attributo "ID_Utente".
- Prenotazione → Aggiunta dell'attributo "ID_Prenotazione".
- Passeggero → Aggiunta dell'attributo "ID_Passeggero".
- Gate → Aggiunta dell'attributo "ID_Gate".

Per le entità Volo Nazionale e Volo Internazionale, si è deciso di usare l'attributo "CodiceVolo" come chiave primaria, in quanto esso identifica univocamente ogni volo. Avendo però due entità distinte, la chiave "CodiceVolo" è stata rinominata in "CodiceVoloN" per i voli nazionali e "CodiceVoloIN" per i voli internazionali.

2.3 Class Diagram Ristrutturato



3 Dizionari

3.1 Dizionario delle Classi

Classe	Descrizione	Attributi
Utente	Rappresenta un utente del sistema, che può rappresentare un utente generico o un amministratore.	• ID_Utente (integer, PK): Chiave primaria surrogata che identifica univocamente un utente. • Username (varchar[20]): Nome utente dell'utente. • Password (varchar[255]): Password dell'utente. • isAdmin (boolean): Flag che indica se un utente appartiene all'amministrazione o meno.
Passeggero	Rappresenta un passeggero, con i suoi dati personali.	• ID_Passeggero (integer, PK): Chiave primaria surrogata che identifica univocamente un passeggero. • Nome (varchar[30]): Nome del passeggero. • Cognome (varchar[30]): Cognome del passeggero. • Documento (varchar[50]): Documento di identità del passeggero.
Prenotazione	Rappresenta una prenotazione effettuata da un passeggero per un volo.	• ID_Prenotazione (integer, PK): Chiave primaria surrogata che identifica univocamente una prenotazione. • CodicePrenotazione (varchar[20]): Codice della prenotazione effettuata. • PostoAssegnato (varchar[10]): Posto assegnato al passeggero. • StatoPrenotazione (varchar[20]): Stato della prenotazione (confermata, cancellata, in attesa). • ID_Passeggero (integer): Chiave esterna che fa riferimento al passeggero associato alla prenotazione. • ID_Utente (integer): Chiave esterna che fa riferimento all'utente che ha effettuato la prenotazione. • CodiceVolo (varchar[5]): Chiave esterna che fa riferimento al volo associato alla prenotazione.

Classe	Descrizione	Attributi
Volo Nazionale	Rappresenta un volo su territorio nazionale.	• CodiceVoloN (varchar[5], PK): Chiave primaria che identifica univocamente un volo nazionale. • Compagnia (varchar[10]): Identifica la compagnia aerea del volo. • Partenza (varchar[20]): Città di partenza del volo. • Destinazione (varchar[20]): Città di arrivo del volo. • RegioneDestinazione (varchar[20]): Indica la regione di destinazione del volo nazionale. • DataOraPartenza (timestamp): Data e ora di partenza del volo. • DataOraArrivo (timestamp): Data e ora di arrivo del volo. • ID_Gate (integer): Chiave esterna che fa riferimento al gate associato al volo.
Volo Internazionale	Rappresenta un volo su territorio internazionale.	• CodiceVoloIN (varchar[5], PK): Chiave primaria che identifica univocamente un volo internazionale. • Compagnia (varchar[10]): Identifica la compagnia aerea del volo. • Partenza (varchar[20]): Città di partenza del volo. • Destinazione (varchar[20]): Città di arrivo del volo. • CodiceNazione (varchar[5]): Rappresenta un codice che indica la nazione di destinazione del volo. • TipoDocumento (varchar[20]): Indica il tipo di documento richiesto per il volo, che può essere il Passaporto, il Visto o la Carta d'Identità. • DataOraPartenza (timestamp): Data e ora di partenza del volo. • DataOraArrivo (timestamp): Data e ora di arrivo del volo. • ID_Gate (integer): Chiave esterna che fa riferimento al gate associato al volo.

Classe	Descrizione	Attributi
Gate	Rappresenta un gate dell'aeroporto, con le sue informazioni.	• ID_Gate (integer, PK): Chiave primaria surrogata che identifica univocamente un gate. • CodiceGate (varchar[5]): Codice nominativo del gate. • Terminale (integer): Identifica il terminale di appartenenza del gate. • StatoGate (varchar[10]): Rappresenta lo stato del gate, che può essere Aperto, Chiuso o Occupato.

3.2 Dizionario delle Associazioni

- **Gestione Profili (Utente - Passeggero)**: Associazione 1 : N. Un *Utente* può creare e gestire molteplici profili *Passeggero* (1..*), tipicamente per familiari o conoscenti. Ogni *Passeggero* deve però appartenere a un solo *Utente* responsabile (1..1).
- **Titolarità (Passeggero - Prenotazione)**: Associazione 1 : N. Un *Passeggero* può figurare come titolare di diverse *Prenotazioni* effettuate nel tempo (1..*). Ogni *Prenotazione* è però emessa a nome di un singolo ed unico *Passeggero* (1..1).
- **Assegnazione Volo (Prenotazione - Volo)**: Associazione 1 : N. Ogni *Prenotazione* garantisce un posto su un unico *Volo Nazionale* o *Internazionale* (1..1). Ciascun *Volo* può ricevere prenotazioni da numerosi passeggeri (1..*).
- **Assegnazione Gate (Volo - Gate)**: Associazione 1 : 1. Un *Volo* viene associato a un *Gate* specifico per le operazioni di imbarco (0..1). Il sistema prevede che, per motivi di sicurezza o logistica immediata, un *Gate* sia dedicato a un solo volo alla volta (0..1).

3.3 Dizionario dei Vincoli

Tipo di Vincolo	Descrizione	Attributi Coinvolti
Vincolo di Integrità Primaria (PK)	Garantisce l'univocità delle righe all'interno di una tabella, identificando ogni record in modo certo e non nullo.	• ID_Utente, ID_Passeggero, ID_Prenotazione, ID_Gate: Identificativi univoci e immutabili per le rispettive entità. • CodiceVoloN, CodiceVoloIN: Codici identificativi univoci per i voli nazionali e internazionali.
Vincolo di Integrità Referenziale (FK)	Assicura la consistenza dei legami tra le tabelle, forzando la corrispondenza tra una chiave esterna e una chiave primaria esistente.	• ID_Utente: Lega i record di Passeggero e Prenotazione all'account utente responsabile. • ID_Passeggero: Associa ogni Prenotazione al profilo anagrafico del viaggiatore. • CodiceVoloN, CodiceVoloIN: Collegano la Prenotazione alla specifica tratta aerea selezionata. • ID_Gate: Associa i Voli (Nazionali o Internazionali) al punto di imbarco assegnato.
Vincolo di Unicità (UNIQUE)	Garantisce che i valori contenuti in una colonna, o in una combinazione di esse, siano unici all'interno di tutta la tabella.	• Username: Impedisce la creazione di più account con lo stesso nome utente. • CodicePrenotazione: Assicura che ogni biglietto emesso abbia un identificativo univoco nel sistema. • Combinazione Volo/Posto: Impedisce l'assegnazione dello stesso posto a più passeggeri sullo stesso volo.
Vincolo di Controllo (CHECK)	Definisce condizioni logiche personalizzate che i dati devono soddisfare per essere ritenuti validi dal sistema.	• Cronologia Orari: Verifica che la data e l'ora di arrivo siano sempre successive a quelle di partenza. • Esclusività della Tratta: Garantisce che una prenotazione faccia riferimento a un solo tipo di volo (Nazionale o Internazionale). • Stato Entità: Limita i valori inseribili per gli stati di Gate e Prenotazione a quelli predefiniti.
Vincolo di Integrità (NOT NULL)	Impone l'obbligatorietà di inserimento per determinati campi, impedendo la presenza di valori nulli dove l'informazione è essenziale.	• Dati Anagrafici: Nome, Cognome e Documento del passeggero. • Dati Autenticazione: Password dell'utente. • Dati Logistici: Città di partenza e destinazione per ogni volo.

4 Modello Logico

Si arriva quindi alla fase di modellazione logica, in cui il modello concettuale viene tradotto in uno schema relazionale.

4.1 Schema Logico

Il seguente schema relazionale rappresenta la struttura logica della base di dati. Nello specifico, le chiavi primarie sono indicate con una singola sottolineatura, mentre le chiavi esterne sono indicate con una doppia sottolineatura.

- **UTENTE** (ID_Utente, Username, Password, isAdmin)
- **PASSEGGERO** (ID_Passeggero, Nome, Cognome, Documento, ID_Utente)
- **GATE** (ID_Gate, CodiceGate, Terminale, StatoGate)
- **VOLO_NAZIONALE** (CodiceVoloN, Compagnia, Partenza, Destinazione, RegioneDestinazione, DataOraPartenza, DataOraArrivo, ID_Gate)
- **VOLO_INTERNAZIONALE** (CodiceVoloIN, Compagnia, Partenza, Destinazione, CodiceNazione, TipoDocumento, DataOraPartenza, DataOraArrivo, ID_Gate)
- **PRENOTAZIONE** (ID_Prenotazione, CodicePrenotazione, PostoAssegnato, StatoPrenotazione, ID_Passeggero, ID_Utente, CodiceVoloN, CodiceVoloIN)

5 Progettazione Fisica

La progettazione fisica consiste nella traduzione dello schema logico in un insieme di comandi SQL per la creazione delle tabelle e dei vincoli nel DBMS PostgreSQL.

5.1 Definizioni delle Tabelle

Di seguito è riportato il codice SQL per la creazione delle tabelle del database.

5.1.1 Tabella Utente

```
1 CREATE TABLE Utente (  
2     ID_Utente SERIAL PRIMARY KEY,  
3     Username VARCHAR(20) UNIQUE NOT NULL,  
4     Password VARCHAR(255) NOT NULL,  
5     isAdmin BOOLEAN NOT NULL  
6 );
```

5.1.2 Tabella Passeggero

```
1 CREATE TABLE Passeggero (  
2     ID_Passeggero SERIAL PRIMARY KEY,  
3     Nome VARCHAR(30) NOT NULL,  
4     Cognome VARCHAR(30) NOT NULL,  
5     Documento VARCHAR(50) NOT NULL,  
6     ID_Utente INTEGER REFERENCES Utente(ID_Utente) ON  
    DELETE CASCADE  
7 );
```

5.1.3 Tabella Prenotazione (con Trigger)

```
1 CREATE TABLE Prenotazione (  
2     ID_Prenotazione SERIAL PRIMARY KEY,  
3     CodicePrenotazione VARCHAR(20) UNIQUE NOT NULL,  
4     PostoAssegnato VARCHAR(10) NOT NULL,  
5     StatoPrenotazione VARCHAR(20) DEFAULT 'In Attesa'  
6     CHECK (StatoPrenotazione IN ('Confermata', '  
    Cancellata', 'In Attesa')),  
7     ID_Passeggero INTEGER NOT NULL REFERENCES PASSEGGERO(  
    ID_Passeggero),  
8     ID_Utente INTEGER NOT NULL REFERENCES UTENTE(  
    ID_Utente),  
9     CodiceVoloN VARCHAR(5) REFERENCES VOLO_NAZIONALE(  
    CodiceVoloN),  
10    CodiceVoloIN VARCHAR(5) REFERENCES  
    VOLO_INTERNAZIONALE(CodiceVoloIN),  
11  
12    -- Vincolo XOR: una prenotazione deve avere o un volo  
    N o un volo IN, non entrambi  
13    CONSTRAINT chk_xor_volo CHECK (
```

```
14      (CodiceVoloN IS NOT NULL AND CodiceVoloIN IS NULL
15      ) OR
16      (CodiceVoloN IS NULL AND CodiceVoloIN IS NOT NULL
17      )
18  );
19
20  --Funzione per controllare la disponibilità del posto
21  CREATE OR REPLACE FUNCTION check_posto_disponibile()
22  RETURNS TRIGGER AS $$
23  BEGIN
24      -- Controlla se esiste già una prenotazione
25      confermata per lo stesso volo e stesso posto
26      IF EXISTS (
27          SELECT 1 FROM PRENOTAZIONE
28          WHERE PostoAssegnato = NEW.PostoAssegnato
29          AND (CodiceVoloN = NEW.CodiceVoloN OR
30              CodiceVoloIN = NEW.CodiceVoloIN)
31          AND StatoPrenotazione = 'Confermata'
32          AND ID_Prenotazione != NEW.ID_Prenotazione
33      ) THEN
34          RAISE EXCEPTION 'Errore: Il posto % é già
35          stato occupato per questo volo.', NEW.PostoAssegnato;
36      END IF;
37
38      RETURN NEW;
39  END;
40
41  CREATE OR REPLACE TRIGGER trg_check_posto
42  BEFORE INSERT OR UPDATE ON PRENOTAZIONE
43  FOR EACH ROW EXECUTE FUNCTION check_posto_disponibile
44  ();
45
46  CREATE OR REPLACE FUNCTION check_coerenza_documenti()
47  RETURNS TRIGGER AS $$
48  DECLARE
49      tipo_richiesto_dal_volo VARCHAR;
50      tipo_posseduto_dal_passeggero VARCHAR;
51  BEGIN
52      -- Se il volo non é internazionale, non c'è
53      bisogno di fare nulla
54      IF NEW.CodiceVoloIN IS NULL THEN
55          RETURN NEW;
```

```

50         END IF;
51
52         -- Ottenimento del tipo di documento richiesto.
53         SELECT TipoDocumento INTO tipo_richiesto_dal_volo
54         FROM VOLO_INTERNAZIONALE
55         WHERE CodiceVoloIN = NEW.CodiceVoloIN;
56
57         -- Ottenimento del tipo di documento del
58         passeggero.
59         SELECT Documento INTO
60         tipo_posseduto_dal_passeggero
61         FROM PASSEGGERO
62         WHERE ID_Passeggero = NEW.ID_Passeggero;
63
64         -- Confronto tra i due dati, per verificare se il
65         passeggero é idoneo.
66         IF tipo_richiesto_dal_volo !=
67         tipo_posseduto_dal_passeggero THEN
68             RAISE EXCEPTION 'Impossibile prenotare: il
69             volo richiede %, ma il passeggero ha %.',
70             tipo_richiesto_dal_volo,
71             tipo_posseduto_dal_passeggero;
72         END IF;
73
74         RETURN NEW;
75     END;
76
77     CREATE OR REPLACE TRIGGER trg_coerenza_documenti
78     BEFORE INSERT OR UPDATE ON PRENOTAZIONE
79     FOR EACH ROW EXECUTE FUNCTION
80     check_coerenza_documenti();

```

5.1.4 Tabella Volo Nazionale

```

1  CREATE TABLE Volo_Nazionale (
2  CodiceVoloN VARCHAR(5) PRIMARY KEY,
3  Compagnia VARCHAR(10) NOT NULL,
4  Partenza VARCHAR(20) NOT NULL,
5  Destinazione VARCHAR(20) NOT NULL,
6  RegioneDestinazione VARCHAR(20) NOT NULL,
7  DataOraPartenza TIMESTAMP NOT NULL,
8  DataOraArrivo TIMESTAMP NOT NULL,
9  ID_Gate INTEGER REFERENCES GATE(ID_Gate),

```

```
10      CONSTRAINT chk_date_voloN CHECK (DataOraArrivo >
      DataOraPartenza)
11 );
```

5.1.5 Tabella Volo Internazionale

```
1  CREATE TABLE Volo_Internazionale (
2  CodiceVoloIN VARCHAR(5) PRIMARY KEY,
3  Compagnia VARCHAR(10) NOT NULL,
4  Partenza VARCHAR(20) NOT NULL,
5  Destinazione VARCHAR(20) NOT NULL,
6  CodiceNazione VARCHAR(5) NOT NULL,
7  TipoDocumento VARCHAR(20) CHECK (TipoDocumento IN ('
Passaporto', 'Visto', 'Carta Identit ')),
8  DataOraPartenza TIMESTAMP NOT NULL,
9  DataOraArrivo TIMESTAMP NOT NULL,
10 ID_Gate INTEGER REFERENCES GATE(ID_Gate),
11 CONSTRAINT chk_date_voloIN CHECK (DataOraArrivo >
DataOraPartenza)
12 );
```

5.1.6 Tabella Gate

```
1  CREATE TABLE Gate (
2  ID_Gate SERIAL PRIMARY KEY,
3  CodiceGate VARCHAR(5) UNIQUE NOT NULL,
4  Terminale INTEGER NOT NULL,
5  StatoGate VARCHAR(10) CHECK (StatoGate IN ('Aperto',
'Chiuso', 'Occupato'))
6  );
7
8  --Non si pu  inserire un volo ad un gate chiuso.
9  CREATE OR REPLACE FUNCTION check_gate_chiuso()
10 RETURNS TRIGGER AS $$
11 DECLARE
12     stato_attuale VARCHAR(10);
13 BEGIN
14     SELECT StatoGate INTO stato_attuale FROM Gate WHERE
ID_Gate = NEW.ID_Gate;
15     IF stato_attuale = 'Chiuso' THEN
16         RAISE EXCEPTION 'Impossibile assegnare un volo al
gate %,   chiuso.', NEW.ID_Gate;
```

```

17     END IF;
18     RETURN NEW;
19 END;
20
21 --Trigger per i voli nazionali
22 CREATE OR REPLACE TRIGGER trg_check_gate_nazionale
23 BEFORE INSERT OR UPDATE ON Volo_Nazionale
24 FOR EACH ROW
25 EXECUTE FUNCTION check_gate_chiuso();
26
27 --Trigger per i voli internazionali
28 CREATE OR REPLACE TRIGGER trg_check_gate_internazionale
29 BEFORE INSERT OR UPDATE ON Volo_Internazionale
30 FOR EACH ROW
31 EXECUTE FUNCTION check_gate_chiuso();

```

5.2 Trigger e altre Funzioni

5.2.1 Trigger per la gestione delle prenotazioni di un volo cancellato

Questo trigger, è pensato per gestire una rimozione di un volo. In particolare, quando un volo viene cancellato, il sistema non cancella le prenotazioni associate al volo, ma cambia lo stato della prenotazione in "Cancellata", in modo da mantenerne traccia. Inoltre, se il volo cancellato occupava un gate, il trigger farà sì che il gate venga liberato.

```

1 CREATE OR REPLACE FUNCTION gestione_cancellazione_volo()
2 RETURNS TRIGGER AS $$
3 BEGIN
4     -- Annulla tutte le prenotazioni collegate al volo
5     -- cancellato
6     -- Operiamo sui record cancellati usando OLD
7     UPDATE PRENOTAZIONE
8     SET StatoPrenotazione = 'Cancellata'
9     WHERE CodiceVoloN = OLD.CodiceVoloN OR CodiceVoloIN =
10     OLD.CodiceVoloIN;
11
12     -- Se il volo aveva un Gate assegnato, riportiamo il
13     Gate a "Aperto"
14     IF OLD.ID_Gate IS NOT NULL THEN
15         UPDATE GATE
16         SET StatoGate = 'Aperto'

```

```
14         WHERE ID_Gate = OLD.ID_Gate;
15
16         RAISE NOTICE 'Gate % riaperto a seguito della
cancellazione del volo.', OLD.ID_Gate;
17     END IF;
18
19     RAISE NOTICE 'Tutte le prenotazioni per il volo %
sono state annullate.',
20         COALESCE(OLD.CodiceVoloN, OLD.
CodiceVoloIN);
21
22     RETURN OLD;
23 END;
24 $$ LANGUAGE plpgsql;
25
26 -- Applichiamo il trigger a entrambe le tabelle voli
27 CREATE TRIGGER trg_cancella_volo_naz
28 AFTER DELETE ON VOLO_NAZIONALE
29 FOR EACH ROW EXECUTE FUNCTION gestione_cancellazione_volo
    ();
30
31 CREATE TRIGGER trg_cancella_volo_int
32 AFTER DELETE ON VOLO_INTERNAZIONALE
33 FOR EACH ROW EXECUTE FUNCTION gestione_cancellazione_volo
    ();
```